# MODULUS

## Retail Operations Platform

### Implementation Roadmap

Version 1.0

# 1. Introduction

This document defines the implementation sequence for the Modulus Retail Operations Platform. It organises the build into nine sequential phases, each with a defined goal, a set of tasks, measurable exit criteria, and a clear statement of dependencies on prior phases.

No phase begins until the preceding phase has met all of its exit criteria. This sequencing ensures that shared foundations are stable before business logic is built on top of them, and that integration is validated continuously rather than deferred to the end.

# 2. Phase Summary

| Phase | Title | Primary Output |
|---|---|---|
| 0 | Monorepo Scaffold | Turborepo workspace with four apps and three shared packages |
| 1 | Shared Packages | UI component library, auth context, shared types, config |
| 2 | Shell Application | Login, layout, routing, Module Federation host |
| 3 | Inventory MFE | Product catalogue management, Module Federation remote |
| 4 | Orders MFE | Order management and status workflow, Module Federation remote |
| 5 | Analytics MFE | BI dashboard with charts, Module Federation remote |
| 6 | Integration | All four apps composed and verified end to end |
| 7 | CI/CD Pipelines | GitHub Actions workflows for all four applications |
| 8 | Playwright E2E | Full E2E suite with reporting and Slack alerting |
| 9 | Polish | Responsive layout, error boundaries, empty states, README |

# 3. Phase Definitions

## Phase 0: Monorepo Scaffold

Goal: Establish the monorepo workspace. No application code is written in this phase. The output is a runnable scaffold that all subsequent phases build into.

### Tasks

- Initialise the root package.json with workspaces configuration covering apps/* and packages/*
- Install and configure Turborepo with a turbo.json pipeline defining build, dev, lint, and test tasks
- Scaffold four application directories under apps/: shell, inventory, orders, analytics
- Scaffold three package directories under packages/: ui, auth, types
- Add a packages/config directory with shared tsconfig.base.json, .eslintrc.base.js, and tailwind.config.base.js
- Configure each app with its own package.json, tsconfig.json (extends base), and .eslintrc.js (extends base)
- Verify that turbo run build executes across all workspaces without errors
- Initialise the Git repository and add .gitignore covering node_modules, dist, .turbo, and .env files

### Exit Criteria

- turbo run build completes without errors across all workspaces
- turbo run lint completes without errors across all workspaces
- Directory structure matches the specification in the System Design Document exactly
- No application-specific code exists yet: apps contain only configuration files

### Dependencies

None. This is the first phase.

## Phase 1: Shared Packages

Goal: Build the shared packages that all four applications will consume. These packages define the visual language, authentication interface, and type contracts for the entire platform.

### Tasks

- packages/types: define Product, Order, OrderStatus, OrderItem, StatusEvent, Customer, AnalyticsSummary, and User types
- packages/types: define MSW handlers for all API endpoints with responses matching the declared types

- packages/types: seed data generation script producing 120 products, 500 orders, 60 customers, and analytics aggregates
- packages/auth: implement AuthContext with user, token, login, and logout
- packages/auth: implement useAuth hook
- packages/auth: implement ProtectedRoute component for use in the shell router
- packages/ui: implement Button (primary, secondary, destructive, ghost variants with loading state)
- packages/ui: implement Table (sortable, paginated, configurable columns)
- packages/ui: implement Badge (status variants)
- packages/ui: implement Modal (focus trap, keyboard dismissal)
- packages/ui: implement SlideOver panel
- packages/ui: implement Skeleton placeholder
- packages/ui: implement Card, Input, Select, Textarea, Toast, EmptyState
- Write Vitest unit tests for all shared components covering render, interaction, and accessibility

### Exit Criteria

- All shared package types compile under TypeScript strict mode with zero errors
- All shared UI components render correctly in isolation
- Vitest unit test coverage is above 80 percent for packages/ui and packages/auth
- MSW handlers return correctly typed responses for all defined endpoints
- Seed data generation script runs without errors and produces valid data matching all type definitions

### Dependencies

Phase 0 must be complete. All workspaces must be resolvable.

## Phase 2: Shell Application

Goal: Build the shell: authentication, persistent layout, routing, and Module Federation host configuration. The shell must be functional as a standalone application before any remote is connected.

### Tasks

- Configure Webpack 5 with ModuleFederationPlugin as host, consuming inventory, orders, and analytics remotes
- Implement the login page with form validation, error handling, and redirect on success
- Implement JWT token storage in memory via AuthContext
- Implement role-based route protection using ProtectedRoute from packages/auth
- Implement the persistent layout: top navigation bar with user avatar and logout, sidebar with section links

- Implement the router with routes: /login, /inventory, /orders, /analytics, / (redirects to /inventory)
- Implement dynamic remote loading with React.lazy and Suspense for each route
- Implement error boundary per remote route to isolate load failures
- Implement loading skeleton displayed while remote module loads
- Implement Access Denied page for role-restricted navigation attempts
- Configure remote URLs via environment variables: VITE_INVENTORY_URL, VITE_ORDERS_URL, VITE_ANALYTICS_URL
- Configure Vercel deployment for modulus-shell project

### Exit Criteria

- Login, logout, and redirect flows work correctly for all three credential sets
- Role-based access correctly blocks navigation to restricted sections
- Remote loading skeleton appears on navigation and resolves when remote loads
- Error boundary renders fallback UI when a remote URL is unreachable (tested by pointing to an invalid URL)
- Shell builds and deploys to Vercel independently with no remote applications running
- No TypeScript errors under strict mode

### Dependencies

Phase 1 must be complete. packages/auth and packages/ui must be buildable.

## Phase 3: Inventory MFE

Goal: Build the Inventory micro-frontend as a fully functional standalone application and as a Module Federation remote consumable by the shell.

### Tasks

- Configure Webpack 5 with ModuleFederationPlugin as remote, exposing ./App
- Implement the product table: paginated (20 per page), searchable by name or SKU, filterable by category and stock status
- Implement stock level badge: green for more than 20 units, amber for 5 to 20 units, red for fewer than 5 units
- Implement add product SlideOver with fields: name, SKU, category, price, stock quantity, image URL, and Zod validation
- Implement edit product SlideOver pre-populated from selected row
- Implement deactivate and reactivate product with confirmation modal
- Implement category management: add and rename categories via modal
- Connect all operations to MSW handlers via fetch calls to /api/products and /api/categories
- Implement optimistic UI updates for all mutations
- Implement Zustand store for local UI state (selected product, open panels, active filters)

- Configure Vercel deployment for modulus-inventory project

### Exit Criteria

- All CRUD operations complete successfully against MSW handlers
- Search, filter, and pagination work correctly across the full seeded data set
- Stock level badges render with correct colour for all three stock ranges
- Optimistic updates apply immediately and roll back correctly on simulated error
- Application runs as a standalone Vite dev server on port 3001
- Application loads correctly as a remote in the shell at the /inventory route
- No TypeScript errors under strict mode

### Dependencies

Phase 2 must be complete. The shell must be able to resolve and load remotes.

## Phase 4: Orders MFE

Goal: Build the Orders micro-frontend as a fully functional standalone application and as a Module Federation remote.

### Tasks

- Configure Webpack 5 with ModuleFederationPlugin as remote, exposing ./App
- Implement the order table: paginated, searchable by order ID or customer email, filterable by status
- Implement order detail view: customer information, line items table, order total, and status timeline
- Implement status update workflow: status selector, confirmation modal, optimistic update, rollback on error
- Enforce valid status transitions in the UI: pending to processing, processing to shipped, shipped to delivered, any to cancelled
- Implement order timeline component showing all status events with timestamps in chronological order
- Implement customer lookup: search by email returns all orders for that customer
- Connect all operations to MSW handlers via fetch calls to /api/orders and /api/customers
- Implement Zustand store for local UI state
- Configure Vercel deployment for modulus-orders project

### Exit Criteria

- All order status transitions enforce the defined valid transition rules
- Invalid transitions are disabled in the UI and rejected by the MSW handler
- Order detail view renders correctly for all order states in the seed data
- Customer lookup returns correct results across the seeded customer set

- Application runs as a standalone Vite dev server on port 3002
- Application loads correctly as a remote in the shell at the /orders route
- No TypeScript errors under strict mode

### Dependencies

Phase 2 must be complete. packages/types must define all Order-related types.

## Phase 5: Analytics MFE

Goal: Build the Analytics micro-frontend as a read-only BI dashboard and as a Module Federation remote.

### Tasks

- Configure Webpack 5 with ModuleFederationPlugin as remote, exposing ./App
- Implement metric cards: total revenue, total orders, average order value, return rate
- Implement daily revenue LineChart for the last 30 days using Recharts
- Implement order volume by status PieChart using Recharts
- Implement top 10 products by revenue horizontal BarChart using Recharts
- Implement regional sales breakdown horizontal BarChart using Recharts
- Implement date range selector that re-fetches all chart data on change
- Connect all data fetching to MSW handlers via fetch calls to /api/analytics
- Implement loading skeleton for each chart section during fetch
- Configure Vercel deployment for modulus-analytics project

### Exit Criteria

- All four charts render correctly with seeded data
- Date range selector re-fetches and re-renders all charts without page reload
- Loading skeletons appear during fetch and resolve correctly
- Application runs as a standalone Vite dev server on port 3003
- Application loads correctly as a remote in the shell at the /analytics route
- No TypeScript errors under strict mode

### Dependencies

Phase 2 must be complete. packages/types must define all Analytics types.

## Phase 6: Integration

Goal: Verify the complete composed application end to end. All four applications must be running simultaneously and all cross-application behaviours must be validated.

### Tasks

- Run all four applications concurrently using the turbo dev command
- Verify authentication state propagates correctly from the shell into each remote via useAuth
- Verify that navigation between sections loads the correct remote without a full page reload
- Verify the error boundary behaviour by intentionally pointing a remote URL to an invalid address
- Verify role-based access control by logging in with each credential set and confirming correct section access
- Verify that standalone remote dev servers work independently without the shell running
- Perform a full manual walkthrough of the user journey defined in the Playwright E2E Test Plan
- Fix any integration issues before proceeding

### Exit Criteria

- Full manual walkthrough completes without errors on all three credential sets
- All four remote applications load within the shell without React singleton violations
- Error boundary renders correctly when a remote is taken offline
- Role restrictions function correctly for all three roles
- No console errors in the browser during any part of the walkthrough

### Dependencies

Phases 3, 4, and 5 must all be complete and deployed to Vercel.

## Phase 7: CI/CD Pipelines

Goal: Implement GitHub Actions pipelines for all four applications. Each pipeline runs independently on push to its application directory and deploys to its corresponding Vercel project.

### Tasks

- Create .github/workflows/shell.yml triggered on push to apps/shell/**
- Create .github/workflows/inventory.yml triggered on push to apps/inventory/**
- Create .github/workflows/orders.yml triggered on push to apps/orders/**
- Create .github/workflows/analytics.yml triggered on push to apps/analytics/**
- Each workflow runs stages in sequence: checkout, install, lint, typecheck, unit test, build, deploy to Vercel
- Configure Vercel CLI deployment in each workflow using VERCEL_TOKEN, VERCEL_ORG_ID, and VERCEL_PROJECT_ID secrets

- Shell workflow adds a final stage after deploy: trigger the Playwright E2E suite
- All pipelines fail fast: a failure in any stage stops subsequent stages
- All pipelines report job status (pass/fail) on the pull request checks surface
- Configure branch protection on main requiring all pipeline checks to pass before merge

### Exit Criteria

- A push to apps/shell triggers only the shell pipeline, not others
- A push to apps/inventory triggers only the inventory pipeline, not others
- A deliberate lint error in any application fails the pipeline at the lint stage
- A deliberate TypeScript error fails the pipeline at the typecheck stage
- A deliberate failing unit test fails the pipeline at the unit test stage
- A successful pipeline deploys to the correct Vercel project
- Branch protection blocks merge when any pipeline check fails

### Dependencies

Phase 6 must be complete. All four applications must be deployed to Vercel with stable URLs.

## Phase 8: Playwright E2E Suite

Goal: Implement the full Playwright end-to-end test suite covering the complete realistic operations workflow. Configure reporting, failure alerting, and deployment gating.

### Tasks

- Install Playwright and configure playwright.config.ts targeting the production shell URL
- Implement the full 14-step user journey as defined in the Playwright E2E Test Plan document
- Configure Playwright to run in Chromium only for portfolio demonstration (expandable to cross-browser)
- Implement page object models for: LoginPage, InventoryPage, OrdersPage, AnalyticsPage, Shell
- Configure HTML reporter output to the playwright-report/ directory
- Add a GitHub Actions job in shell.yml to run the Playwright suite after successful deploy
- Configure the Playwright job to upload the HTML report to GitHub Pages on every run
- Configure the Playwright job to send a Slack webhook notification on test failure
- Slack notification must include: failed test name, step number, affected MFE, and a direct URL to the GitHub Pages report
- Configure the shell pipeline to fail if any Playwright test fails, blocking the deployment from being considered successful

### Exit Criteria

- Full 14-step user journey passes without errors on the production composed application

- A deliberately broken test fails the CI pipeline and blocks the shell deployment status
- HTML report is published to GitHub Pages and accessible via a public URL after every CI run
- Slack notification fires on test failure with all required fields: test name, step, MFE, and report URL
- Slack notification does not fire on a passing run
- Page object models encapsulate all selector logic: no raw selectors appear in test files

### Dependencies

Phase 7 must be complete. The shell CI/CD pipeline must be functional and triggering on push.

## Phase 9: Polish

Goal: Harden the application for portfolio presentation. This phase adds no new features. It improves reliability, edge case handling, and documentation.

### Tasks

- Audit all pages for empty state handling: any list or table with no results must show an EmptyState component
- Audit all async operations for loading state handling: no operation should leave the UI in a blank or frozen state
- Audit all error states: API errors must display a user-facing message, not a raw error object
- Verify responsive layout at 1024px, 1280px, and 1440px viewport widths (desktop-first)
- Run Lighthouse on the shell and each remote: target Performance score above 85, Accessibility score above 90
- Fix any accessibility issues identified by Lighthouse: ARIA labels, focus management, keyboard navigation
- Write the README.md at the monorepo root covering: project overview, architecture summary, running locally, environment variables, and links to all four deployed URLs
- Add architecture diagram to the README (ASCII or Mermaid)
- Record a short screen capture walkthrough of the full user journey for the portfolio entry
- Final manual smoke test of all four applications on their production Vercel URLs

### Exit Criteria

- All list and table views have empty state handling
- No async operation leaves the UI in an unresponsive state
- Lighthouse Performance score is above 85 on all four deployed applications
- Lighthouse Accessibility score is above 90 on all four deployed applications
- README is complete with all required sections and correct URLs
- Final manual smoke test passes without errors on production URLs
- All six documentation files are complete and final

### Dependencies

Phase 8 must be complete. The full CI/CD and E2E pipeline must be functioning.

## 4. Development Principles

The following principles govern implementation decisions across all phases:

- TypeScript strict mode is enforced from Phase 0. No any types are permitted in application code.
- No phase may be considered complete until all its exit criteria are met. Partial completion does not permit advancement.
- Shared package changes must not break existing consumers. Run turbo run build after any change to packages/*.
- Every API call uses types from packages/types. No inline type definitions for API responses are permitted in application code.
- No console.log statements are committed. Use a named logger or remove debug output before committing.
- Environment variables are validated at startup using Zod. An application with missing required variables must fail loudly on boot, not silently at runtime.

## 5. Risk Register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Module Federation version conflicts between remotes | Medium | High | Pin React and react-dom to identical patch versions in all four applications from Phase 0 |
| MSW service worker scope conflicts across remotes | Low | High | Initialise MSW only in the shell; remotes must not initialise their own MSW instance |
| Turborepo cache stale after shared package changes | Medium | Medium | Run turbo run build --force after any change to packages/* |
| Vercel free tier build minute limits | Low | Medium | Use turbo run build --filter to build only changed applications locally before pushing |
| Playwright tests flaky due to async remote loading | Medium | Medium | Use page.waitForSelector on a stable element inside each MFE before asserting |